Database Systems Lab
Deployment

Christian Rauch

(changed: August 27, 2025)

JOHANNES GUTENBERG
UNIVERSITÄT MAINZ
JG|U

Local or Cloud Deployment

- *A deployment is not required for the practicum.*
- This presentation provides a minimal and incomplete configuration for demonstration purposes.
- For local deployment, uWSGI is the preferred solution.
- For cloud deployment, Kubernetes offers comprehensive container orchestration with:
  - Automatic scaling and intelligent load balancing
  - Self-healing infrastructure with automatic container recovery
  - Seamless rolling updates with zero downtime
  - Built-in service discovery and DNS resolution

Local Deployment

Install uWSGI (or an alternative) using the package manager.

```
sudo apt install uwsgi
sudo apt install uwsgi-plugin-python3
```

Create a WSGI configuration for your application.

```
[uwsgi]
plugins       = python3
socket        = 127.0.0.1:5010
chdir         = /srv/x
protocol      = http
wsgi-file     = app.py
callable      = app
processes     = 4
threads       = 2
buffer-size   = 32768
stats         = 127.0.0.1:9191
virtualenv    = /srv/x/.venv
http-timeout  = 86400
uid           = chris
```

Local Deployment

Run the WSGI server with this configuration to test the setup.

```
uwsgi --ini /srv/x/myapp.ini
```

Open localhost:5010 in a browser and check if it works. Kill the process afterwards.

Copy the configuration to the apps-available directory and create a symbolic link in the apps-enabled directory.

```
sudo ln -s /etc/uwsgi/apps-available/myapp.ini
    /etc/uwsgi/apps-enabled
```

Enable and (re)start.

```
sudo systemctl enable uwsgi
sudo systemctl start uwsgi
```

You would typically install a reverse proxy (e.g., Nginx) and set it up to forward requests to WSGI server.

Cloud Deployment

Kubernetes is a container orchestration platform for deploying applications.
Key concepts:

- Pods: Deploy containers
- Services: Expose apps
- Deployments: Manage replicas
- ConfigMaps/Secrets: Store config/data
- Ingress: External access
- Namespaces: Isolate resources

Cloud Deployment

Deployment configuration to manage Pods (scaling, updates, rollback, self-healing):

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: flask-webapp-deployment
5    labels:
6      app: flask-webapp
7  spec:
8    replicas: 3
9    selector:
10     matchLabels:
11       app: flask-webapp
12   template:
13     metadata:
14       labels:
15         app: flask-webapp
16     spec:
```

Cloud Deployment

```
containers:
- name: flask-webapp
  image: my-flask-app:v1.0
  ports:
  - containerPort: 5000
  env:
  - name: FLASK_ENV
    value: "production"
  - name: DATABASE_URL
    value: "mysql://user:pass@db:3306/db"
  resources:
    requests:
      memory: "128Mi"
      cpu: "100m"
    limits:
      memory: "256Mi"
      cpu: "200m"
```

Cloud Deployment

Service configuration:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: webapp-service
5  spec:
6    selector:
7      app: webapp
8    ports:
9    - port: 80
10     targetPort: 5000
11   type: ClusterIP
```

Cloud Deployment

Ingress configuration for external access:

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: webapp-ingress
5  spec:
6    rules:
7    - host: myapp.example.com
8      http:
9        paths:
10       - path: /
11         pathType: Prefix
12         backend:
13           service:
14             name: webapp-service
15             port:
16               number: 80
```

Conclusion

Thank you for your attention!